

C++ Preprocessor

The preprocessor in C++ is used for processing the code before it is compiled by the compiler. It does many tasks such as including files, conditional compilation, using macros, etc. The preprocessor also allows the developers to select which portions of code should be included or excluded.

The code processed by the preprocessor is called expanded code and is generally saved with a ".i" file extension.

Preprocessor Directives in C++

In C++, the preprocessor directives are special commands that are used to instruct the preprocessor. It begins with a '#' symbol and tells the preprocessor to modify source code before compilation. There are different preprocessor directives in C++ for different operations.

The below table lists frequently used preprocessor directives:

Directive	Purpose
#include	Links a header file in the source code.
#define	Creates a symbolic or macro constant.
#undef	Deletes a macro that has already been defined.
#ifdef / #ifndef	Compilation that is conditional on the presence or absence of a macro.
#if / #elif / #else / #endif	Compilation that is conditional based on some expression.
#error	Halts the compilation process and produces an error notice.
#warning	During compilation, a warning notice is shown.
#pragma	Provide the compiler specific instructions.

#include

The **#include** preprocessor directive is used to include the contents of one file into the current one, use the #include directive. Header files are often included using this.

The syntax is as follows:

```
#include <header_file_name>
```

header_file_name is the name of the header file to be included. And the <> tell the compiler to include the header files from the source directory.

Or

```
#include "header_file_name"
```

Here “” tells the compiler to include the header files from the directory the source file is currently in.

Implement the following in a new project, within a file titled **HelloWorld.cpp**.

```
// C++ program to demonstrate use of #include preprocessor  
// directives  
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "Hello World! << endl;  
    return 0;  
}
```

#define

The #define preprocessor directive is used to define macros. Macro names are symbolic and may be used to represent constant values or short bits of code. Using #define preprocessor makes our code more readable and easily maintainable as we can replace numbers and code snippets with a meaningful name.

The syntax is as follows:

```
#define macro_name value
```

Implement the following in a new project, within a file titled **Macro.cpp**.

```
// C++ program to demonstrate the use of #define
// preprocessor directive.

#include <iostream>
using namespace std;

#define PI 3.14159
#define findSquare(x) (x * x)

int main(){
    double radius = 5.0;
    double area = PI * findSquare(radius);

    cout << "Area of circle: " << area;
    return 0;
}
```

#undef

The #undef preprocessor directive is used to undefined a previously defined macro (defined using #define). It is mainly used in the case when we want to redefine an existing macro or eliminate a macro definition associated with it from the code.

The syntax is as follows:

```
#undef macro_name
```

Implement the following in a new project, within a file titled **MacroTwo.cpp**.

```
// C++ program to demonstrate the use of #undef preprocessor
// directive.

#include <iostream>
using namespace std;
#define MAX_VALUE 100

int main(){
    cout << "Max value: " << MAX_VALUE << endl;

    // using undef to change MAX_VALUE
    #undef MAX_VALUE
    #define MAX_VALUE 200

    cout << "Max value: " << MAX_VALUE;

    return 0;
}
```

#ifdef and #ifndef

The `#ifdef` and `#ifndef` are preprocessor directives that are used for conditional compilation. `#ifndef` verifies that a macro is not defined, `#ifdef` verifies that a macro is defined.

The syntax is as follows:

```
#ifdef macro_name  
    // Code to be executed if macro_name is defined  
#ifndef macro_name  
    // Code to be executed if macro_name is not defined  
#endif
```

`#ifdef` and `#ifndef` are often used with the `#endif` directive to include or exclude portions of code based on whether a certain macro is defined or not.

Implement the following in a new project, within a file titled **ConditionalCompilation.cpp**.

```
// C++ program to demonstrate the use of #ifdef and #ifndef
// preprocessor directives.

#include <iostream>
using namespace std;
#define DEBUG
#define PI 3.14

int main(){
    // to check if DEBUG is defined
#ifdef DEBUG
    cout << "Debug mode is ON" << endl;
#else
    cout << "Debug mode is OFF" << endl;
#endif

    // to check if PI is defined
#ifndef PI
    cout << "PI is not defined" << endl;
#else
    cout << "PI is defined" << endl;
#endif

    return 0;
}
```

#if, #elif, #else, and #endif Directives (Conditional Directives)

The #if, #elif, #else, #endif, and #error are conditional preprocessor directives these are used for conditional compilation. These are used to include or exclude a code based on some conditions specified.

The syntax is as follows:

```
#if constant_expr

    // Code to be executed if constant_expression is true

#elif another_constant_expr

    // Code to be excuted if another_constant_expression is true

#else

    // Code to be excuted if none of the above conditions are true

#endif
```

Implement the following in a new project, within a file titled **ConditionalDirectives.cpp**.

```
// C++ program to demonstrates the use of #if, #elif, #else,
// and #endif  preprocessor directives.
#include <iostream>
using namespace std;

// defining PI
#define PI 3.14159

int main(){
    #if defined(PI)
        cout << "PI is defined" << endl;
    #elif defined(SQUARE)
        cout << "PI is not defined" << endl;
    #else
        #error "Neither PI nor SQUARE is defined"
    #endif

    return 0;
}
```

#error

The `#error` directive is a preprocessor directive that is used to print a custom error message for compilation error. If any condition is not met or any particular requirement is not satisfied we can stop the compilation process using `#error`.

The syntax is as follows:

`#error error_message`

Where *error_message* is the custom error message that you want to print when the `#error` is encountered.

Implement the following in a new project, within a file titled **Error.cpp**.

```
// C++ program to demonstrate the use #error preprocessor
// directives.

#include <iostream>
using namespace std;

// not defining PI here
// #define PI 3.14159

int main(){
    #if defined(PI)
        cout << "PI is defined" << endl;
    #else
        #error "Neither PI nor SQUARE is defined"
    #endif

    return 0;
}
```

#warning

The #warning preprocessor directive is used to generate a warning message during compilation. We can write custom warning messages generally used for any informational or debugging purpose. The compiler prints the warning message in console, to given an alert about any required condition or decisions that are made during the preprocessing stage.

The syntax is as follows:

`#warning message`

Here, *message* is any custom message that you want to print as an alert.

Implement the following in a new project, within a file titled **Warning.cpp**.

```
// C++ program to demonstrate the use of #warning
// preprocessor directive.
#include <iostream>
using namespace std;

// not defining it to trigger the warning
// #define PI 3.14

#ifndef PI
#warning "PI is not defined!"
#endif

int main(){
    cout << "Hello World" << endl;
    return 0;
}
```


#pragma

The `#pragma` directive is a compiler-specific instructions. Special instructions to the compiler are provided using the `#pragma` directive. It may be used, for instance, to alter compiler parameters or silence warnings. behavior and supported pragmas can vary between different compilers, as they are compiler-specific.

The syntax is as follows:

`#pragma directive`

Commonly Used #pragma Flags

- `#pragma once`: used to include guard for header files.
- `#pragma message`: used to print custom messages at the time of compilation.
- `#pragma warning`: used to control warning behavior (like enable or disable warnings).
- `#pragma optimize`: used to control optimization settings (manage optimization level).
- `#pragma comment`: used to include some additional information in the object file(or specify linker options).

Implement the following in a new project, within a file titled **Pragma.cpp**.

```
// C++ program to demonstrate the use of pragma preprocessor
// directive.

#include <iostream>
using namespace std;

#pragma once

// Defining PI to trigger a pragma message during
// compilation
#define PI 3.14

// to set aggressive optimization level
#pragma optimize("O3")

int main(){
#ifdef PI
#pragma message("YES! PI is defined.")
#endif
    cout << "In main function!\n";
    return 0;
}
```

In summary, C++ preprocessor directives is a helpful tool to insert the code before actual compilation it improve the modularity, maintainability, and customisation of the code. It uses directives with a # symbol to give instructions to compiler. It allows the developers to modify and manage the code for various conditions and manage the compilation process.

References

- <https://www.geeksforgeeks.org/cpp-preprocessors-and-directives/>